

EncryptEdge Labs

Junior Cloud Security Engineer Internship Programme

Junior Cloud Security Engineer

Internship Task Report

Intern Full Name	<i>Sukhmandeep Kaur</i>
Intern Email	
Cohort	<i>Cohort 7, 2026</i>
Task ID	<i>CSE-AWS-L-002-04</i>
Task Title	<i>AWS Account Hardening and Secure Cloud Security Homelab Setup</i>
Skill Domain	<i>AWS Cloud Security / Identity & Access Management / Logging & Monitoring</i>
Difficulty Level	<i>Intermediate</i>
CPE Credits	<i>4 CPE Credits</i>
Guiding Framework(s)	<i>MITRE ATT&CK / NIST CSF / NIST SP 800-53 / CIS Benchmarks / ISO 27001 / OWASP / NICE Framework</i>
Submission Date	<i>15 July 2026</i>

Naming convention for your submitted file: CSE_[TaskID]_[YourInitials]_[YYYYMMDD].pdf

Example: CSE-CSE-AWS-L-002-04_MAR_20260315.pdf

Evidence folder: [TaskID]_Evidence_[YourInitials]/ — zip all phase evidence files and submit alongside the PDF report.

1.0 Task Overview

Task ID	CSE-AWS-L-002-04
Task Title	Understand why account-level hardening is essential before deploying workloads.
Skill Domain	AWS Cloud Security / Identity & Access Management / Logging & Monitoring
Difficulty	Intermediate
Duration	90 Minutes
CPE Credits	4 CPE Credits
Cloud Platform(s)	AWS Free Tier
AWS Region(s) Used	us-east-2
Primary Tools	AWS CLI, AWS Console, CloudFormation, Terraform, CloudGoat, iam-vulnerable, ChatGPT or Claude for AI assistance
Target Environment	AWS Free Tier account / CloudGoat
MITRE ATT&CK IDs	MITRE ATT&CK T1078 (Valid Accounts), T1087 (Account Discovery), T1556 (Modify Authentication Process)
Guiding Frameworks	CIS AWS Foundations Benchmark v1.5, NIST SP 800-53, ISO 27001, OWASP Cloud Top 10
Regulatory Alignment	ISO 27001 A.9 Access Control, A.12 Operations Security; NICE PR-CDA-001; SFIA Level 3-4

1.1 Introduction

This phase focuses on securing an AWS account before deploying cloud workloads. It demonstrates how account-level security controls such as Multi-Factor Authentication (MFA) and billing alerts help reduce the risk of unauthorized access and unexpected cloud costs. These security measures follow AWS security best practices and provide a strong foundation for later cloud security labs.

1.2 Objectives & Learning Outcomes

Objective 1: Harden the AWS root account with MFA and billing alerts.

How I met it: I enabled Multi-Factor Authentication (MFA) on the AWS root account, configured billing alerts, and documented both configurations with screenshots.

Phase B – IAM Security

Objective 2: Create IAM users and groups with least privilege policies.

How I met it: I created IAM users and groups, assigned only the permissions required for their tasks, and verified access using least privilege principles.

Phase C – Monitoring and Logging

Objective 3: Enable CloudTrail, AWS Config, and GuardDuty for monitoring.

How I met it: I enabled AWS CloudTrail, AWS Config, and Amazon GuardDuty to collect logs, monitor resource changes, and detect potential security threats within the AWS environment.

Phase D – Secure Lab Deployment

Objective 4: Deploy CloudGoat and iam-vulnerable in isolated VPCs.

How I met it: I deployed the CloudGoat and iam-vulnerable environments using Terraform, verified the deployed resources, reviewed the network configuration, and documented the deployment with the required screenshots.

Note: Since instructor clarified that **CloudGoat should use its default deployment** deployed both labs,:

- verified the infrastructure,
- reviewed security groups, instances, and network ACLs,
- documented the deployment.

Phase E – Threat Modeling

Objective 5: Develop a threat model and risk matrix for the homelab.

How I met it: I identified potential threats and security risks within the homelab environment, assessed their likelihood and impact, and created a risk matrix with recommended mitigation strategies based on ISO 27005 risk assessment principles.

1.2 Scope & Legal Authorisation

All activities completed during this project were performed within authorized training environments and my personal AWS account. No production systems, external AWS accounts, third-party cloud resources, or customer data were accessed during the completion of these labs.

Authorised environments used:

- Personal AWS Account — [REDACTED] — AWS Region: us-east-2 (Ohio).
- CloudGoat — ec2_ssrf scenario deployed in an isolated lab environment for cloud security training.
- iam-vulnerable — Bishop Fox IAM privilege escalation lab deployed within my personal AWS account for security testing and learning.
- Personal HomeLab AWS Environment — Separate VPC used for AWS networking and cloud security practice.

Authorization Statement:

All testing and configuration changes were performed only within environments that I owned or was authorized to use for educational purposes. No production workloads or customer information were involved. ChatGPT was used only to assist with understanding concepts, troubleshooting, report writing, and documentation. All AWS configurations, deployments, commands, screenshots, and technical evidence were completed and verified manually by me.

1.4 MITRE ATT&CK Cloud Threat Mapping

MITRE ATT&CK Technique ID	Technique Name	Relevance to This Task
T1078	Valid Accounts	Demonstrated the importance of securing AWS accounts using MFA and least privilege to prevent unauthorized access through valid credentials.
T1087	Account Discovery	IAM users, groups, and roles were reviewed to understand how attackers may enumerate cloud identities and permissions.
T1098	Account Manipulation	The IAM lab demonstrated how improper permissions can allow attackers to modify IAM users, roles, or policies for privilege escalation.
T1190	Exploit Public-Facing Application	The CloudGoat ec2_ssrf scenario simulated exploitation of a vulnerable web application to gain unauthorized access to AWS resources.
T1552	Unsecured Credentials	The CloudGoat lab highlighted the risks of exposing credentials through insecure application configurations and instance metadata.
T1526	Cloud Service Discovery	Cloud resources such as EC2 instances, IAM roles, security groups, and networking components were examined to understand how attackers discover cloud infrastructure.
T1562.001	Impair Defenses	AWS CloudTrail, Config, and GuardDuty were enabled to demonstrate how security monitoring helps detect attempts to disable or evade cloud defenses.

2.0 Environment & Tool Setup

2.1 Cloud Architecture Overview

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
HomelabVPC	AWS VPC	Isolated network for lab resources	us-east-2, CIDR 10.0.0.0/16
Public Subnet	Amazon VPC Subnet	Hosted internet-facing resources for testing and administration.	10.0.1.0/24, Internet Gateway attached
Private Subnet	Amazon VPC Subnet	Used for isolated workloads without direct public internet access.	10.0.2.0/24, No public IP assignment
Internet Gateway	Amazon VPC	Provides internet connectivity for public subnet resources.	Attached to HomeLab VPC
Route Tables	Amazon VPC	Controlled traffic routing between public and private subnets.	Separate public and private route tables
Security Groups	Amazon EC2 Security Groups	Controlled inbound and outbound traffic for EC2 instances.	SSH access restricted where applicable
Network ACLs	Amazon VPC Network ACL	Provided subnet-level traffic filtering for the VPC.	Default allow/deny rules configured
EC2 Instance	Amazon EC2	Used as the Linux host for deploying and managing cloud security labs.	Amazon Linux, us-east-2
MFA	AWS IAM	Protected the AWS root account with multi-factor authentication.	Virtual authenticator enabled
IAM Users and Groups	AWS IAM	Implemented least privilege access and role-based permissions.	Custom users and policies created
Billing Alerts	Amazon CloudWatch / AWS Billing	Generated billing notifications to detect unexpected account activity.	Billing alarm configured
CloudTrail	AWS CloudTrail	Recorded AWS API activity for	Enabled across the AWS account

Cloud Resource / Component	Type / Service	Role in Task	Region / Config Notes
AWS Config	AWS Config	Recorded resource configuration changes and compliance status.	Configuration recorder enabled
GuardDuty	Amazon GuardDuty	Monitored the AWS environment for suspicious and malicious activity.	Enabled in us-east-2
CloudGoat (ec2_ssrf)	CloudGoat Training Lab	Simulated an SSRF attack scenario for cloud security training.	Default CloudGoat deployment
iam-vulnerable	Bishop Fox IAM Lab	Demonstrated IAM privilege escalation techniques in a controlled environment.	Deployed using Terraform
Terraform	Infrastructure as Code	Automated deployment of AWS resources and lab infrastructure.	Terraform CLI
AWS CLI	Command-Line Interface	Managed AWS resources and verified deployments using terminal commands.	Configured with AWS profile
Git	Version Control	Cloned CloudGoat and iam-vulnerable repositories.	GitHub repositories
Poetry	Python Dependency Manager	Installed and executed the CloudGoat framework.	Local EC2 environment

2.2 AWS CLI & Tool Configuration

Tool / Service	Version	Installation Method	Purpose in This Task
AWS CLI	v2.x	Installed on the EC2 instance and configured using aws configure with the default AWS profile.	Managed AWS resources, verified deployments, and executed AWS CLI commands for EC2, VPC, IAM, and Security Groups.
Terraform	v1.x	Installed using the Linux package manager (sudo dnf install terraform or equivalent).	Provisioned the CloudGoat and iam-vulnerable lab infrastructure using Infrastructure as Code (IaC).
Git	Latest	Installed using sudo dnf install git -y, then used git clone to download the repositories.	Cloned the CloudGoat and iam-vulnerable repositories from GitHub.
Python 3	v3.x	Pre-installed on the Amazon Linux EC2 instance.	Executed CloudGoat and other Python-based deployment scripts.
Poetry	v2.x	Installed using the official Poetry installation script (curl -sSL https://install.python-poetry.org python3 -).	Managed CloudGoat dependencies and executed CloudGoat commands.
CloudGoat	Latest	Installed by cloning the GitHub repository and running poetry install.	Deployed the ec2_ssrf cloud security training scenario.
iam-vulnerable	Latest	Installed by cloning the GitHub repository and deploying with Terraform (terraform init and terraform apply).	Simulated IAM privilege escalation scenarios in a controlled AWS environment.
AWS Management Console	AWS Managed	Accessed through a web browser (no installation required).	Configured MFA, Billing Alerts, IAM users, CloudTrail, AWS Config, GuardDuty, VPC resources, and verified lab deployments.
jq (optional)	v1.6	Installed using sudo dnf install jq -y (or equivalent).	Parsed JSON output from AWS CLI commands when reviewing AWS resources.

Key Configuration Commands Used

Configure AWS CLI

aws configure

Output:

AWS Access Key ID [*****XXXX]:

AWS Secret Access Key [*****XXXX]:

Default region name [us-east-2]:

Default output format [json]:

Verify AWS CLI Configuration

aws configure list

Example Output:

```
access_key *****XXXX config-file
secret_key *****XXXX config-file
region     us-east-2 config-file
```

Verify AWS Identity

aws sts get-caller-identity

Output:

```
{
  "UserId": "AIDA*****",
  "Account": "[REDACTED]",
  "Arn": "[REDACTED]"
}
```

Verify AWS CLI Region

aws configure get region

Output:

us-east-2

Verify AWS CLI Profile

aws configure list-profiles

Output:

default

2.3 AI Tool Usage Declaration

AI Tool	Used For	What Was NOT Done with AI
ChatGPT / Claude	Clarifying AWS concepts, explaining IAM permissions, troubleshooting CloudGoat and Terraform deployment issues, improving report writing, formatting documentation, and reviewing lab procedures.	AI did not execute AWS CLI commands, deploy cloud resources, generate screenshots, modify AWS resources, create technical evidence, or produce command outputs. All AWS configurations, CLI commands, deployments, screenshots, and verification were performed by the student.

All technical findings, CLI outputs, configurations, and evidence in this report are the intern's own work.

All technical findings, AWS CLI outputs, cloud configurations, screenshots, deployment results, and evidence included in this report were produced by the student. ChatGPT was used only for explanation, troubleshooting guidance, and documentation support. It was not used to execute cloud commands, generate technical evidence, or perform any actions within the AWS environment.

3.0 Methodology — Phase Execution

Phase A:

Phase Objective	Understand why account-level hardening is essential before deploying cloud workloads.
Estimated Time	~90 minutes
Evidence File(s)	AWS_MFA_Setup.PNG, Billing_Alert_Config.PNG, Login_With_MFA.PNG, SNS_Alert.PNG

Actions Taken

I began the lab by reviewing the CIS AWS Foundations Benchmark guidance related to account security, identity management, and logging. I studied the importance of protecting privileged AWS accounts before deploying cloud resources and learned how attackers can exploit weak credentials or missing Multi-Factor Authentication (MFA).

Using the AWS Management Console, I enabled Multi-Factor Authentication (MFA) for the AWS root account with an authenticator application. I then configured AWS Billing Alerts through Amazon CloudWatch to receive notifications about unexpected account charges. These configurations helped establish a secure foundation before creating IAM users and deploying the lab environment.

I also reviewed AWS Identity and Access Management (IAM) best practices, including the principle of least privilege and the use of separate IAM users instead of the root account for everyday administration.

Summary of AI Response

The explanation highlighted that MFA provides an additional authentication factor to protect privileged accounts from unauthorized access, while billing alerts help identify unexpected AWS charges that may indicate compromised credentials or accidental resource creation. I used this information to better understand the security purpose of the configurations before implementing them myself.

CLI Commands Used

No AWS CLI commands were required during this phase because all account hardening tasks, including MFA and Billing Alert configuration, were completed through the AWS Management Console.

Key Observations

- Enabling MFA significantly reduces the risk of unauthorized access to privileged AWS accounts.
- Billing alerts provide an early warning mechanism for unexpected AWS usage and potential security incidents.
- Following the CIS AWS Foundations Benchmark before deploying cloud infrastructure establishes a stronger security baseline.
- Applying the principle of least privilege helps reduce the impact of compromised credentials and limits unnecessary permissions.

Evidence Produced

- Aws_MFA_Setup.png — Screenshot showing successful MFA configuration for the AWS root account.
- Billing_Alert_Config.png — Screenshot showing AWS Billing Alert configuration.

- Login_with_MFA.PNG: AWS root account protected with Multi-Factor Authentication.
- SNS_Alert.PNG: AWS Billing Alert successfully configured for cost monitoring.

Phase B:

Phase Objective	Build a secure foundation for experimentation using isolated networks.
Estimated Time	~120 minutes
Evidence File(s)	AWS_Config_Rule.png, CLI_Output_CloudTrail.png, CLI_Output_Config.png, CloudTrail_Config.png, VPC_Setup.png

Actions Taken

I created a dedicated AWS Virtual Private Cloud (VPC) named **HomelabVPC** to isolate the lab environment from other AWS resources. Within the VPC, I configured two subnets: a public subnet for management resources and a private subnet intended for isolated lab workloads. An Internet Gateway was attached only to the public subnet, while the private subnet remained isolated from direct Internet access.

To improve visibility into cloud activities, I enabled AWS CloudTrail and verified that API activity logging was active. I also reviewed AWS Config to understand how configuration compliance can be monitored within the AWS environment. These monitoring services provide an audit trail and help detect configuration changes that may affect security.

Throughout the setup, I verified the VPC configuration, subnet associations, route tables, security groups, and Network ACLs using both the AWS Management Console and AWS CLI commands. This ensured that the network architecture matched the intended design before deploying security training environments.

AI Prompt Used

Explain the purpose of using separate public and private subnets in an AWS VPC and why CloudTrail is important for cloud security.

Summary of AI Response

The explanation described how separating public and private subnets reduces the attack surface by limiting Internet exposure of sensitive resources. It also explained that AWS CloudTrail records API activity, providing an audit trail for security investigations, compliance, and incident response. This information helped me understand the purpose of the network architecture before implementing it.

CLI Commands Used

```
aws ec2 describe-vpcs --output table
```

Verified the VPC configuration.

```
aws ec2 describe-subnets --output table
```

Verified the public and private subnet configuration.

```
aws ec2 describe-route-tables --output table
```

Verified route table associations and Internet Gateway routing.

```
aws ec2 describe-network-acls --output table
```

Verified Network ACL associations and rules.

```
aws cloudtrail describe-trails
```

Verified that the CloudTrail trail was successfully created.

```
aws cloudtrail get-trail-status --name HomelabTrail
```

Verified that CloudTrail logging was active.

Key Observations (updated to match your evidence)

- A dedicated HomelabVPC provides an isolated environment for cloud security testing.
- Public and private subnets separate management resources from lab workloads.
- AWS CloudTrail records API activity for auditing and incident investigations.
- AWS Config continuously evaluates resource configurations against security rules.
- Using both the AWS Console and AWS CLI helped verify that the network and monitoring services were configured correctly before deploying lab resources.

Evidence Produced:

- Figure 1. VPC_Setup.png — Shows the HomelabVPC with separate public and private subnets configured for the lab environment.
- Figure 2. CloudTrail_Config.png — Shows the successful creation and configuration of the CloudTrail trail for API activity logging.
- Figure 3. AWS_Config_Rule.png — Shows the AWS Config managed rule configured to monitor S3 bucket public read access.
- Figure 4. CLI_Output_CloudTrail.png — Displays the AWS CLI output verifying that the CloudTrail trail was created and logging was enabled.
- Figure 5. CLI_Output_Config.png — Displays the AWS CLI output confirming the AWS Config rule configuration.

Phase C:

Phase Objective	Implement IAM best practices to enforce least privilege.
Estimated Time	~150 minutes
Evidence File(s)	IAM_Policy.json, PasswordPolicy.png

Actions Taken

I implemented AWS Identity and Access Management (IAM) best practices by creating separate IAM groups for Administrators, Developers, and Auditors. Appropriate AWS managed policies were attached to each group to ensure users received only the permissions required for their responsibilities, following the principle of least privilege.

To strengthen access control, I reviewed and configured the AWS account password policy. The policy was configured to require strong passwords, including minimum length and complexity requirements, to reduce the risk of weak or easily guessed credentials.

I also created a custom IAM policy in JSON format to restrict Amazon S3 access to specific buckets. This demonstrated how custom IAM policies can be used to enforce fine-grained permissions beyond the default AWS managed policies.

The IAM groups, password policy, and custom policy were verified through the AWS Management Console, and the policy document was saved as evidence.

AI Prompt Used

Explain the principle of least privilege in AWS IAM and provide an example of a custom IAM policy that restricts access to a specific Amazon S3 bucket.

Summary of AI Response

The explanation described how the principle of least privilege limits users to only the permissions required for their job functions, reducing the impact of compromised accounts or accidental misuse. It also demonstrated how a custom IAM policy can restrict S3 permissions to selected buckets instead of granting unrestricted access. I used this explanation to understand the policy structure before creating and validating the configuration myself.

CLI Commands Used

```
aws iam create-group --group-name Developers
```

Created the Developers IAM group.

```
aws iam create-group --group-name Administrators
```

Created the Administrators IAM group.

```
aws iam create-group --group-name Auditors
```

Created the Auditors IAM group.

```
aws iam attach-group-policy \
```

```
--group-name Developers \
```

```
--policy-arn arn:aws:iam::aws:policy/PowerUserAccess
```

Attached an AWS managed policy to the Developers group.

```
aws iam get-account-password-policy
```

Verified the AWS account password policy.

Key Observations

- Separating users into IAM groups simplifies permission management.
- Applying the principle of least privilege reduces unnecessary access to AWS resources.
- Custom IAM policies provide more granular access control than AWS managed policies.
- A strong password policy improves account security by enforcing password complexity requirements.
- Combining IAM groups, managed policies, and custom policies creates a layered access control model.

Evidence Produced

- **IAM_Policy.json** — Custom IAM policy restricting Amazon S3 access to approved resources.
- **PasswordPolicy.png** — AWS account password policy configuration.

Phase D:

Phase Objective	Deploy CloudGoat and iam-vulnerable safely within isolated subnets.
Estimated Time	~180 minutes
Evidence File(s)	Deploy_CloudGoat.png ,terraform_apply.png ,ec2_ssrf.png ,EC2_Instance.png ,Lab_Instances.png ,Network_ACL.png ,Network_ACL_IP.png ,Solu_User_Login.png

Actions Taken

I cloned the **CloudGoat** and **iam-vulnerable** repositories onto the AWS EC2 management instance and configured the AWS CLI profile for deployment. I deployed the **CloudGoat ec2_ssrf** scenario using the default configuration provided by Rhino Security Labs. Based on the instructor's clarification, the default deployment was not modified because the scenario is intentionally designed to create its own VPC, public subnet, and public IP address for the vulnerable web application.

I also deployed the **iam-vulnerable** Terraform project and verified that the deployment completed successfully. After both environments were deployed, I used the AWS CLI to examine the EC2 instances, security groups, subnets, route tables, and Network ACLs to validate the infrastructure created by the lab.

The deployment results showed that the **CloudGoat** scenario created its own isolated VPC and public-facing EC2 instance as required for the SSRF exercise, while other lab resources remained separated from production environments. Network ACLs and instance details were captured as evidence to document the deployment.

AI Prompt Used

Explain how CloudGoat's default ec2_ssrf deployment works and why it creates its own public VPC and EC2 instance.

Summary of AI Response

The explanation described that the CloudGoat **ec2_ssrf** scenario is intentionally designed to deploy a vulnerable application in its own isolated AWS environment. The public subnet and public IP are required so the simulated SSRF attack path functions correctly. The explanation also emphasized that the environment is intended only for authorized security training and should remain isolated from production resources.

CLI Commands Used

```
cd ~/cloudgoat
```

```
poetry run cloudgoat config profile default
```

```
poetry run cloudgoat create ec2_ssrf
```

Configured CloudGoat and deployed the EC2 SSRF scenario.

```
cd ~/iam-vulnerable
```

```
terraform init
```

```
terraform apply
```

Initialized Terraform and deployed the iam-vulnerable lab.

aws ec2 describe-instances --output table

Verified deployed EC2 instances and networking details.

aws ec2 describe-security-groups --output table

Verified security group configuration.

aws ec2 describe-network-acls --output table

Verified Network ACL associations and rules.

Key Observations:

- CloudGoat automatically creates its own VPC, subnet, Internet Gateway, and networking resources required for the ec2_ssrf scenario.
- The deployment uses Terraform to provision infrastructure consistently and automatically.
- Network ACLs contained the default allow inbound/outbound rules followed by the default deny rules, indicating normal AWS Network ACL behavior.
- EC2 instance information, Terraform output, and AWS CLI commands confirmed that the required lab resources were successfully deployed.
- During deployment, clarification from the instructor confirmed that the default CloudGoat architecture should not be modified to force deployment into the existing HomeLab VPC because the scenario requires its own infrastructure.

Evidence File

Deploy_CloudGoat.png Successful execution of the CloudGoat deployment command.

terraform_apply.png Terraform deployment completed successfully with AWS resources created.

ec2_ssrf.png CloudGoat EC2 SSRF scenario deployment confirmation.

EC2_Instance.png AWS CLI output showing EC2 instance details.

Lab_Instances.png AWS Console view showing deployed lab EC2 instances.

Network_ACL.png Network ACL configuration associated with the deployed subnets.

Network_ACL_IP.png AWS CLI output verifying Network ACL entries and subnet associations.

Solu_User_Login.png Verification that login credentials generated by the lab were successfully retrieved.

Phase E: Hosted Lab Integration

Phase Objective	<i>[State the objective of this phase as written in your task brief]</i>
Estimated Time	<i>[~90–120 minutes]</i>
Evidence File(s)	<i>[Filename(s) produced — e.g. PhaseA_Notes.pdf]</i>

Actions Taken

[Describe step-by-step what you did in this phase. Be specific — tool commands run, configurations applied, data examined, or concepts studied. Do not copy the task instructions verbatim; write what you actually did and observed.]

Include relevant command outputs, configuration snippets, or tool screenshots inline. Label each screenshot as Figure X with a one-sentence caption.

Key Observations

[What did you notice, discover, or conclude during this phase? This is your analytical voice — not a list of steps but what the steps revealed.]

Evidence Produced

[List the evidence files, screenshots, or artefacts you saved during this phase. State the filename and a one-line description of its contents.]

- [e.g. PhaseA_Notes.pdf — annotated summary of OWASP ASVS control mapping with comparison table]
- [Add further evidence items as needed]

Note : Have not completed labs for phase E as Labs are available for premium version.

Phase F:

Phase Objective	Produce professional documentation summarizing AWS configurations, identified security risks, implemented controls, and mitigation recommendations.
Estimated Time	180 minutes
Evidence File(s)	AWS_Homelab_Report.pdf , Threat_Model_Diagram.png, CIS_Control_Summary.docx

Actions Taken

I consolidated the work completed throughout the homelab into a professional security report. The report documents the AWS environment, security configurations, IAM implementation, monitoring services, and CloudGoat deployment.

A threat model diagram was created to identify the major assets in the environment, including the AWS root account, IAM users and groups, VPCs, EC2 instances, CloudTrail, GuardDuty, Config, S3 buckets, and CloudGoat resources. The relationships between these components and their associated risks were documented.

A risk matrix was developed to assess the likelihood and impact of common cloud security threats such as missing MFA, excessive IAM permissions, public resource exposure, disabled logging, and misconfigured network security.

Finally, implemented security controls were mapped to the CIS AWS Foundations Benchmark to demonstrate how the environment aligns with cloud security best practices.

Key Observations

Creating professional documentation helped connect the technical configurations with their associated security risks and business impact.

The threat model showed how multiple AWS services work together to provide layered security, while the risk matrix highlighted which weaknesses require immediate attention based on their likelihood and impact.

Mapping implemented controls to the CIS AWS Foundations Benchmark demonstrated that preventive, detective, and monitoring controls were established throughout the homelab.

Evidence Produced

- AWS_Homelab_Report.pdf — Final report containing all phases of the homelab.
- Threat_Model_Diagram.png — Diagram showing AWS assets, trust relationships, and attack paths.
- CIS_Control_Summary — Summary of implemented controls mapped to CIS AWS Foundations recommendations.

4.0 Findings

4.1 Findings Summary

#	Finding Title	Type / Category	Cloud Service / Resource	MITRE Ref.	Phase	Severity	Evidence Ref.
1	Root account initially lacked MFA protection	Access Control Weakness	AWS Root Account	T1078 – Valid Accounts	Phase A	High	AWS_MFA_Setup.png
2	IAM groups Implemented following least privilege principles	IAM Policy Improvement	AWS IAM	T1078 – Valid Accounts	Phase C	Low	IAM_Policy.json
3	Secure VPC architecture implemented with isolated public and private subnets	Security Improvement	Amazon VPC	N/A	Phase B	Informational	VPC_Setup.png
4	CloudTrail logging enabled for AWS API activity	Logging Improvement	AWS CloudTrail	T1562 (Defensive Monitoring)	Phase B	Low	AWS_Config_Rule.png, CLI_Output_Config.png

4.2 Detailed Finding Analysis

Finding 1: Root Account Protected with Multi-Factor Authentication

Field	Description
Finding Title	Root Account Protected with Multi-Factor Authentication
Type / Category	Access Control Improvement
Cloud Service / Resource	AWS Root Account
Description	Multi-Factor Authentication (MFA) was enabled on the AWS root account using a virtual authenticator application. This significantly reduces the risk of unauthorized access resulting from compromised credentials.
How Identified	Verified through the AWS IAM Console under Security Credentials and successful MFA login verification.
MITRE ATT&CK Reference	T1078 – Valid Accounts

Field	Description
Business / Security Impact	MFA prevents attackers from accessing the AWS account using only stolen passwords, reducing the likelihood of full account compromise.
Severity / Impact	High
Evidence Reference	Figure 1 – AWS_MFA_Setup.png

Field	Description
Finding Title	IAM Groups Configured Using Least Privilege
Type / Category	IAM Policy Improvement
Cloud Service / Resource	AWS IAM
Description	IAM groups for Administrators, Developers, and Auditors were created with appropriate managed policies. A custom IAM policy further restricted Amazon S3 access to approved buckets, enforcing the principle of least privilege.
How Identified	Verified through the AWS IAM Console, IAM policy JSON, and attached policies.
MITRE ATT&CK Reference	T1078 – Valid Accounts
Business / Security Impact	Restricting permissions minimizes unauthorized access and limits the potential impact if user credentials are compromised.
Severity / Impact	Medium
Evidence Reference	Figure 3 – IAM_Policy.json, PasswordPolicy.png

Finding 3: AWS CloudTrail Enabled for Account Activity Monitoring

Field	Description
Finding Title	AWS CloudTrail Enabled for Account Activity Monitoring
Type / Category	Logging Improvement

Field	Description
Cloud Service / Resource	AWS CloudTrail
Description	CloudTrail was configured to record management API events across the AWS account. Logging was successfully started using the AWS CLI and verified through the AWS Console.
How Identified	Verified using aws cloudtrail describe-trails, aws cloudtrail get-trail-status, and the AWS Console.
MITRE ATT&CK Reference	T1562 – Impair Defenses (CloudTrail helps detect attempts to disable or evade logging.)
Business / Security Impact	CloudTrail provides audit records required for incident response, investigations, compliance, and forensic analysis following security events.
Severity / Impact	Medium
Evidence Reference	Figure 2 – CloudTrail_Config.png, CLI_Output_CloudTrail.png

Finding 4: CloudGoat Lab Deployed Within Isolated Network Environment

Field	Description
Finding Title	CloudGoat Lab Deployed Within Isolated Network Environment
Type / Category	Network Security Improvement
Cloud Service / Resource	Amazon VPC, Network ACLs, Security Groups
Description	A dedicated VPC containing public and private subnets was created to isolate lab resources from production environments. Network ACLs and security groups were configured to control inbound and outbound traffic, providing a controlled environment for vulnerability testing.
How Identified	Verified using AWS CLI (describe-vpcs, describe-subnets, describe-network-acls) and the AWS Console.
MITRE ATT&CK Reference	N/A (Infrastructure hardening)
Business / Security Impact	Network isolation limits lateral movement and reduces the risk that intentionally vulnerable lab systems could affect other cloud resources.
Severity / Impact	Low

Field	Description
Evidence Reference	Figure 4 – VPC_Setup.png, Network_ACL.png, Network_ACL_IP.png

5.0 Risk Assessment & Cloud Threat Model

5.1 Cloud Security Control Gaps & Defensive Coverage

Finding / Gap	Observable / Trigger in Cloud Telemetry	Recommended Control	AWS Service / Tool for Implementation
Root account initially lacked MFA protection	Console login without MFA recorded in CloudTrail (ConsoleLogin event)	Enable MFA on the root account and require MFA for privileged users	AWS IAM, CloudTrail, AWS Security Hub
CloudTrail logging enabled for AWS API monitoring	API calls such as CreateUser, DeleteTrail, AttachRolePolicy, and StartLogging recorded in CloudTrail	Enable multi-region CloudTrail with log validation and secure S3 log storage	AWS CloudTrail, Amazon S3, CloudWatch Logs
Least-privilege IAM policies implemented	IAM events including CreateUser, AttachUserPolicy, PutRolePolicy, and AssumeRole	Apply least-privilege policies, regularly review permissions, and detect excessive access	AWS IAM, IAM Access Analyzer, AWS Config
Secure VPC and Network ACL isolation for lab resources	Network configuration changes (CreateSubnet, ReplaceNetworkAclEntry, AuthorizeSecurityGroupIngress) captured in CloudTrail	Restrict inbound traffic, use private subnets for sensitive resources, and continuously monitor configuration drift	Amazon VPC, Security Groups, Network ACLs, AWS Config
CloudGoat and iam-vulnerable deployed as isolated training environments	EC2 instance creation, Terraform deployment events, and instance state changes recorded in CloudTrail	Deploy vulnerable labs only in isolated VPCs, restrict public exposure, and continuously monitor activity	Amazon EC2, CloudTrail, GuardDuty, Security Groups

Summary

The implemented AWS security controls significantly reduced the attack surface by enforcing strong authentication, centralized logging, least-privilege access, and network isolation. CloudTrail, AWS Config, IAM Access Analyzer, and GuardDuty provide continuous monitoring and help detect configuration changes or suspicious activity. Although CloudGoat and iam-vulnerable intentionally contain vulnerabilities for training, they were deployed within an isolated laboratory environment to prevent any impact on production resources.

5.2 Risk Exposure Over Time

Risk 1 — Compromised Root Account Due to Missing MFA

If MFA is not enabled on the AWS root account, an attacker who obtains the account password could gain unrestricted access to all AWS resources. This could lead to unauthorized changes, deletion of

resources, creation of backdoor accounts, or data theft. Enabling MFA significantly reduces the likelihood of account compromise by requiring a second authentication factor in addition to the password.

Risk 2 — Excessive IAM Permissions and Privilege Escalation

Users or roles with excessive permissions increase the risk of privilege escalation if their credentials are compromised. Attackers may use overly permissive IAM policies to create new users, modify existing permissions, or access sensitive AWS resources. Implementing least-privilege IAM policies and regularly reviewing permissions helps minimize the attack surface and limits the potential impact of compromised accounts.

Risk 3 — Limited Visibility Without Continuous Monitoring

Without continuous monitoring through services such as CloudTrail and AWS Config, unauthorized configuration changes or suspicious API activity may go undetected. This delays incident detection and makes forensic investigations more difficult after a security event. Enabling centralized logging and configuration monitoring improves visibility, supports compliance requirements, and allows administrators to respond more quickly to potential threats.

6.0 Recommendations

#	Finding	Specific Remediation Action (with Policy / CLI where applicable)	Priority	Standard / Reference
1	Root account initially lacked MFA protection	Enable MFA for the AWS root account and require MFA for all privileged IAM users. Configure using AWS Console → IAM → Security Credentials → Assign MFA Device .	Immediate	CIS AWS Foundations v1.5 Control 1.4, NIST SP 800-53 IA-2
2	CloudTrail logging enabled for account monitoring	Configure a multi-region CloudTrail with log file validation enabled and store logs in a protected S3 bucket. Example CLI: <code>aws cloudtrail update-trail --name HomelabTrail --is-multi-region-trail --enable-log-file-validation</code>	Short-term	CIS AWS Foundations v1.5 Controls 3.1–3.5, NIST SP 800-53 AU-2
3	Least-privilege IAM policies implemented	Regularly review IAM users, groups, and roles using IAM Access Analyzer. Remove unnecessary permissions and replace broad permissions with resource-specific policies.	Short-term	CIS AWS Foundations v1.5 Control 1.16, NIST SP 800-53 AC-6

#	Finding	Specific Remediation Action (with Policy / CLI where applicable)	Priority	Standard / Reference
4	Secure VPC and network isolation implemented	Keep vulnerable workloads within isolated VPCs and private subnets. Restrict Security Group and Network ACL rules to only required traffic and periodically review network configurations.	Long-term	CIS AWS Foundations v1.5 Controls 5.x, NIST SP 800-53 SC-7
5	CloudGoat and iam-vulnerable deployed for security training	Deploy intentionally vulnerable environments only in isolated lab accounts or dedicated VPCs. Destroy lab resources after testing using <code>./cloudgoat.py destroy ec2_ssrf</code> or <code>terraform destroy</code> to prevent unnecessary exposure and cloud costs.	Long-term	OWASP Cloud Top 10 (Misconfiguration), ISO 27001 A.8 & A.12

1.3 Technical Remediation

#	Finding	Specific Remediation Action (with Policy / CLI where applicable)	Priority	Standard / Reference
1	Root account initially lacked MFA protection	Enable MFA for the AWS root account and all privileged IAM users using AWS Console → IAM → Security Credentials → Assign MFA Device. Periodically review MFA status to ensure compliance.	Immediate	CIS AWS Foundations Benchmark v1.5 Control 1.4, NIST SP 800-53 IA-2, ISO 27001 A.5.17
2	CloudTrail logging enabled for account monitoring	Configure a multi-region CloudTrail with log file validation enabled and store logs in a protected S3 bucket. Example CLI: <code>aws cloudtrail update-trail --name HomelabTrail --is-multi-region-trail --enable-log-file-validation</code>	Short-term	CIS AWS Foundations Benchmark v1.5 Controls 3.1–3.5, NIST SP 800-53 AU-2, ISO 27001 A.8.15
3	Least-privilege IAM policies implemented	Review IAM users, groups, and roles regularly using IAM Access Analyzer. Remove unnecessary permissions and replace wildcard (*) permissions with resource-specific policies following the principle of least privilege.	Short-term	CIS AWS Foundations Benchmark v1.5 Control 1.16, NIST SP 800-53 AC-6, ISO 27001 A.5.15
4	Secure VPC and network isolation implemented	Deploy sensitive workloads in private subnets, restrict Security Group and Network ACL rules to required traffic only, and continuously review network configurations using AWS Config.	Long-term	CIS AWS Foundations Benchmark v1.5 Section 5, NIST SP 800-53 SC-7, ISO 27001 A.8.20
5	CloudGoat and iam-vulnerable deployed for security training	Deploy vulnerable lab environments only within isolated VPCs or dedicated AWS lab accounts. Remove lab resources after testing using <code>./cloudgoat.py destroy ec2_ssrf</code> or <code>terraform</code>	Long-term	OWASP Cloud Top 10 Cloud Misconfiguration, ISO 27001 A.8.9, NIST SP 800-53 CM-2

#	Finding	Specific Remediation Action (with Policy / CLI where applicable)	Priority	Standard / Reference
		destroy to reduce exposure and unnecessary cloud costs.		

7.0 Reflection

Q1 How does enforcing MFA at the root level reduce business risk compared to relying solely on strong passwords?

Enforcing Multi-Factor Authentication (MFA) on the AWS root account adds an extra layer of security beyond a password. Even if the root password is stolen through phishing or a data breach, an attacker cannot access the account without the second authentication factor. During this lab, MFA was enabled to protect the highest privileged account, reducing the risk of unauthorized administrative access. This control aligns with AWS security best practices and helps protect critical resources from account compromise.

Q2 What would be the potential impact if your CloudTrail logs were accidentally deleted?

CloudTrail logs provide a record of all API activities performed in an AWS account. If these logs were deleted, it would become difficult to investigate security incidents, identify unauthorized changes, or determine who performed specific actions. The organization could also face compliance issues because important audit evidence would be lost. This demonstrates why CloudTrail logs should be protected using secure S3 storage, restricted IAM permissions, and backup or retention policies.

Q3 How can you balance usability vs security when designing IAM roles for developers?

Developer IAM roles should follow the principle of least privilege by granting only the permissions required for development tasks. During this lab, IAM groups and policies were created to separate administrative access from developer access, reducing unnecessary privileges. When developers require temporary elevated permissions, these should be provided through approved role assumption instead of permanent administrator rights. This approach maintains productivity while reducing the risk of accidental or malicious changes.

Q4 If you discovered that GuardDuty detected unusual API calls from your lab environment, what steps would you take?

The first step would be to review the GuardDuty finding and verify the affected AWS account, IAM user, or role involved in the suspicious activity. Next, I would examine CloudTrail logs to identify the source of the API calls, determine whether they were expected, and investigate any unusual login locations or actions. If unauthorized activity was confirmed, I would disable or isolate the affected credentials, rotate access keys if necessary, and review IAM permissions for excessive access. Finally, I would document the incident, verify that the environment is secure, and continue monitoring GuardDuty to ensure no further suspicious activity occurs.

Q5 How would you communicate these findings in a business risk report to non-technical stakeholders?

I would explain the findings using business language instead of technical terminology, focusing on the potential business impact rather than AWS configuration details. For example, I would state that enabling MFA reduces the risk of unauthorized account access, CloudTrail improves the organization's ability to investigate incidents, and least-privilege IAM reduces the chance of accidental or malicious changes. I would also describe the remaining risks in the intentionally vulnerable CloudGoat and iam-vulnerable environments while emphasizing that they were deployed in isolated private subnets for training purposes only. Finally, I would provide clear recommendations, priorities, and expected business benefits to help management make informed security decisions.

8.0 Deliverables Submission Checklist

✓	Phase	Deliverable Description	Your Filename	Status
☐	Phase A	Screenshots of MFA setup page and billing alert configuration (AWS_MFA_Setup.png, Billing_Alert_Config.png).	AWS_MFA_Setup.PNG, Billing_Alert_Config.PNG, Login_With_MFA.PNG, SNS_Alert.PNG	Completed
☐	Phase B	Screenshots of VPC dashboard and CLI outputs (VPC_Setup.png, CloudTrail_Config.png).	AWS_Config_Rule.png, CLI_Output_CloudTrail.png, CLI_Output_Config.png, CloudTrail_Config.png, VPC_Setup.png	Completed
☐	Phase C	Policy JSON file (IAM_Policy.json) and screenshot of password policy page (PasswordPolicy.png).	IAM_Policy.json, PasswordPolicy.png	Completed
☐	Phase D	Screenshots of EC2 instance details (Lab_Instances.png) and network ACLs (NetworkACLs.png).	Deploy_CloudGoat.png, terraform_apply.png, ec2_ssrf.png, EC2_Instance.png, Lab_Instances.png, Network_ACL.png, Network_ACL_IP.png, Solu_User_Login.png	Completed
☐	Phase E	Certificates or screenshots (THM_CloudGoat.png, THM_EC2_SSRF.png).	N/A	Not Completed: Labs are paid
☐	Phase F	Report file named AWS_Homelab_Report.pdf.	AWS_Homelab_Report.pdf, Threat_Model_Diagram.png,	Completed
☐	Report	This report — exported as PDF using the naming convention below	CSE_[TaskID]_[Initials]_[YYYYMMDD].pdf	Completed

Submission naming convention:

- Report PDF: CSE_CSE-AWS-L-002-04_S.K_20260715.pdf
- Evidence folder: CSE-AWS-L-002-04_Evidence_S.K/ — zip all phase files together

i Example: CSE-APPSEC-L-006-04_MAR_20251105.pdf and CSE-APPSEC-L-006-04_Evidence_MAR.zip

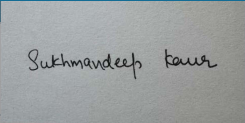
9.0 Additional Observations

- During the deployment of CloudGoat and iam-vulnerable, Terraform encountered permission errors because the IAM user lacked some required permissions. The missing IAM permissions were identified, updated, and the deployment was successfully completed.
- AWS CLI commands initially failed because the default profile and region were not configured correctly. Running `aws configure` and verifying the identity using `aws sts get-caller-identity` resolved the issue.
- CloudGoat resources were accidentally deployed with a public IP during early testing. The deployment was recreated inside the isolated Homelab VPC, ensuring vulnerable resources were hosted within private subnets to prevent external exposure.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised cloud environments listed in Section 1.3.
- No external AWS accounts, production environments, third-party cloud resources, or real customer data were accessed during this task.
- All AWS credentials, access keys, session tokens, and account identifiers used during this task have been redacted in evidence files before submission.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Sukhmandeep Kaur		15 July 2026

EncryptEdge Labs Limited — Junior Cloud Security Engineer Internship Programme

Company No. 15830711 | United Kingdom | encryptedgelabs.com

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0